iApsal

MIGRATION iOS SWIFT → FLUTTER / DART

SYNTHÈSE COMPLÈTE DE PROJET

Architecture · Logique métier · Règles de précision · Plan de développement

Client Proximus NXT Luxembourg	**Application** iApsal — Calculateur de salaires LU
Migration Swift iOS → Flutter Dart (iOS + Android)	**Date** Mars 2026

1. CONTEXTE DU PROJET

1.1 Qu'est-ce qu'iApsal ?

iApsal est une application iOS de calcul de salaires luxembourgeois (Brut→Net et Net→Brut) éditée par Proximus NXT. Elle implémente les paramètres légaux luxembourgeois : cotisations CCSS, barèmes fiscaux, crédits d'impôt, caisse de dépendance, mutualités.

1.2 Objectif de la migration

- Rendre l'app disponible sur Android (actuellement iOS uniquement)
- Moderniser la stack technique vers Flutter/Dart cross-platform
- Préserver la précision exacte des calculs Swift dans tous les cas
- Ajouter 3 axes de features : valeur immédiate, rétention, KPIs Business Analysts

1.3 Valeurs de référence — Tests de conformité obligatoires

Cas	Entrée	Résultat exact iOS	Paramètres	Précision
TEST 1 — Brut→Net	**Brut = 3 500,00 €**	**Net = 2 834,68 €**	2026/01, CI.100, CI=ON, HMC=173	**✓ EXACT** == bit pour bit
TEST 2 — Net→Brut	**Net = 4 000,00 €**	**Brut = 5 668,29 €**	2026/01, CI.100, CI=ON, HMC=173	**✓ EXACT** == bit pour bit

2. PARAMÈTRES LÉGAUX LUXEMBOURGEOIS

2.1 Taux de cotisations par année

Paramètre	2026	2025	2024	2022	Champ
Plafond sécu	13 518,68 €	13 518,68 €	12 854,64 €	10 355,50 €	`PlafondSecu`
CMS (Soins)	2,80%	2,80%	2,80%	2,80%	`CMS = 0.028`
CME (Espèce)	0,25%	0,25%	0,25%	0,25%	`CME = 0.0025`
CMP (Pension)	8,50%	8,00%	8,00%	8,00%	`CMP`
CDEP (Dépend.)	1,40%	1,40%	1,40%	1,40%	`CDEP = 0.014`
ASAC (Acc.)	0,70%	0,70%	0,75%	0,75%	`ASAC`
SAT (Santé trav.)	0,11%	0,11%	0,11%	0,11%	`SAT = 0.0011`
Mutu classe 1	0,70%	0,70%	0,72%	0,60%	`Mutu1`
Mutu classe 2	0,99%	0,99%	1,22%	1,13%	`Mutu2`
Mutu classe 3	1,48%	1,48%	1,76%	1,66%	`Mutu3`
Mutu classe 4	2,64%	2,64%	2,84%	2,98%	`Mutu4`

2.2 Classes d'impôt

- Classe 100 (code interne) = Célibataire — barème le plus chargé
- Classe 200 = Marié / Partenariat — abattement doublé
- Classe 300 = Autres situations — taux intermédiaires

2.3 Crédits d'impôt disponibles

Crédit	Description	Disponibilité
CI	Crédit d'impôt principal — calculé sur brut annualisé × 12	Tous les mois actifs
CI-CO2	Crédit voiture propre — 18€/mois en 2026, 16€ en 2025, 14€ en 2024	2024 et au-delà
CISSM	Crédit salaire social minimum — pour les bas salaires (1 800–3 600 €)	Toutes années
CIM	Crédit impôt monoparental — sur activation bCalculCIM	Toutes années
CIC	Crédit impôt conjoint — 2023 à partir de juin seulement	2023 (juin-déc)
CIE	Crédit impôt épargne — arrêté fin mars 2023	2023 (janv-mars)

3. RÈGLES DE PRÉCISION — CONSIGNES À LA LETTRE

Ces règles sont NON NÉGOCIABLES. Il n'y a aucune tolérance — le résultat Dart doit être identique bit pour bit au résultat iOS. Toute déviation produit des résultats faux immédiatement vérifiables.

⚠️ **NE JAMAIS utiliser .round() natif Dart — utiliser myRound() qui fait floor(x*10^n + 0.5)/10^n. Tolérance : zéro.**

⚠️ **NE JAMAIS mettre le *1000 DANS le int() pour le CISSM — voir règle R3 ci-dessous**

⚠️ **NE JAMAIS utiliser else if dans les barèmes fiscaux — les if doivent être SÉQUENTIELS**

R1 — myRound() : arrondi commercial, pas l'arrondi bancaire

Swift utilise floor(val × 10^n + 0.5) / 10^n. Le .round() Dart fait l'arrondi bancaire (banker's rounding) qui arrondit 0.5 vers le pair le plus proche. Sur des milliers de calculs, la dérive atteint plusieurs euros.

```
// ❌ INTERDIT — arrondi bancaire Dart
(val * 100).round() / 100

// ✅ CORRECT — identique au Swift original
double myRound(double dbVal, int nPlaces) {
  final double dbShift = pow(10.0, nPlaces).toDouble();
  return (dbVal * dbShift + 0.5).floor() / dbShift;
}
```

R2 — Barèmes fiscaux : if séquentiels, jamais else if

Dans BaremeImpot.swift, les if sont séquentiels (pas de else if). Pour certaines valeurs frontières, plusieurs tranches s'évaluent — la dernière qui correspond gagne. Si on met des else if, le résultat d'imposition change.

```
// ❌ INTERDIT — else if coupe la cascade Swift
if (mntimposable > 3060 && mntimposable <= 3250) { pourcentage=24; }
else if (mntimposable > 3250 && mntimposable <= 3445) { pourcentage=26; }

// ✅ CORRECT — if séquentiels comme dans le Swift
if (mntimposable > 3060 && mntimposable <= 3250) { pourcentage=24;
mntdeductible=471.0; }
if (mntimposable > 3250 && mntimposable <= 3445) { pourcentage=26;
mntdeductible=536.075; }
```

R3 — CISSM : Int(val) × 1000, jamais Int(val × 1000)

C'est le bug le plus subtil. Dans CalculSalaire.swift, la formule est :

```
// Swift original :
let iTemp:Int = Int((dMntBrut / dHMC) * Double(HMC_Secu(Annee, Mois))) * 1000
//  ^^^
```

R5 — HMC_Secu : table statique, pas de calcul calendaire

Swift calcule les jours ouvrés dynamiquement via NSCalendar. Flutter n'a pas d'équivalent direct. On utilise une table précalculée identique. Les valeurs sont figées dans hmcSecu() de calcul_salaire.dart.

✓ La table HMC est précalculée pour 2022–2026. Pour ajouter 2027 : compter les jours lundi–vendredi de chaque mois × 8 heures.

R6 — roundUp() : arrondi au centime SUPÉRIEUR

Les crédits d'impôt (CI, CI-CO2, CIC, CIE) utilisent ceil(val × 100) / 100, pas myRound. Ne pas les confondre.

```
double roundUp(double dbVal) => (dbVal * 100).ceil() / 100;

// Utilisé pour : CI, CI-CO2, CIC, CIE, CISSM (palier décroissant)
// NE PAS utiliser pour : cotisations salariales, impôt, dépendance
```

R7 — Net→Brut : myRound(dBrut, 2) APRÈS la boucle

Les paliers fins (+0.01, +0.05...) accumulent des résidus flottants IEEE 754. Sans correction, le brut final vaut 5668.290000000001 au lieu de 5668.29 exact. La solution : appliquer myRound(dBrut, 2) une seule fois après la sortie de la boucle while.

⚠️ **Sans cette ligne, le résultat Net→Brut est faux au niveau flottant même si l'affichage semble correct.**

```
// Dans calculNetBrut(), APRÈS la boucle while, AVANT return true

// ❌ SANS correction — accumulation flottante IEEE 754 :
// 5668.28 + 0.01 = 5668.290000000001
// voSalaire.dBrut == 5668.29  →  false ❌

// ✅ AVEC correction — résultat exact bit pour bit :
voSalaire.dBrut = _maths.myRound(voSalaire.dBrut, 2);
// voSalaire.dBrut == 5668.29  →  true ✅

return true;
```

✓ Preuve : calcul_full(5668.29) == 4000.0 exactement. La boucle converge vers le bon brut, myRound() nettoie le résidu flottant final.

4. ARCHITECTURE FLUTTER

4.1 Principe fondamental — 5 couches strictement séparées

Couche	Dossier	Contenu	Statut Sprint 1
1 — Données	lib/models/	salaire.dart, constantes.dart, simulation.dart	✓ salaire.dart FAIT
2 — Logique	lib/logic/	cls_maths, bareme_impot, impots, calcul_salaire, calcul	✓ 5 fichiers FAITS
3 — État	lib/providers/	simulation_provider, historique_provider, params_provider	Sprint 2
4 — UI	lib/screens/ + widgets/	brut_net_screen, net_brut_screen, resultat_screen...	Sprint 2
5 — Services	lib/services/	storage_service, api_apsal, analytics_service, pdf_service	Sprint 4-5

4.2 Structure complète des fichiers

```
lib/
├── main.dart                          ← Point d'entrée, navigation
├── models/
│   ├── salaire.dart                   ✓ FAIT — objet Salaire complet
│   ├── simulation.dart                Sprint 3 — simulation sauvegardée
│   └── constantes.dart                Sprint 2 — taux par année
├── logic/                             ✓ TOUS FAITS ET VALIDÉS
│   ├── cls_maths.dart                 myRound() + roundUp()
│   ├── bareme_impot.dart              Tranches 2017/2024/2025 (3 classes)
│   ├── impots.dart                    Calcul impôt + solidarité
│   ├── calcul_salaire.dart            Moteur principal complet
│   └── calcul.dart                    Façade BrutNet + NetBrut
├── providers/                         Sprint 2
│   ├── simulation_provider.dart
│   ├── historique_provider.dart
│   └── params_provider.dart
├── screens/                           Sprint 2-3
│   ├── brut_net_screen.dart
│   ├── net_brut_screen.dart
│   ├── resultat_screen.dart
│   ├── historique_screen.dart
│   ├── parametres_screen.dart
│   └── analytics_screen.dart
├── widgets/                           Sprint 2-3
│   ├── salary_slider.dart
│   ├── result_card.dart
│   ├── comparateur_annees.dart
```

```
│   └── export_pdf_button.dart
└── services/                          Sprint 4-5
    ├── storage_service.dart
    ├── api_apsal.dart
    ├── analytics_service.dart
    └── pdf_service.dart
```

4.3 Les 3 règles d'or de l'architecture

Règle 1 — Jamais de calcul dans un Widget

Si brut * 0.028 dans un build() => une erreur.

Tout calcul va dans logic/, appelé depuis un provider.

Les widgets n'affichent que ce que leur provider leur donne.

Règle 2 — Les providers ne connaissent pas l'écran

Un provider expose des données et des méthodes. Point.

Il n'appelle jamais Navigator, n'affiche jamais de dialog.

Il ne connaît aucun widget.

Règle 3 — Les services sont injectés, jamais instanciés

Ne jamais faire ApiApsal().fetchParams() dans un écran.

Le provider reçoit le service en paramètre.

Cela permet de remplacer le vrai service par un faux en test.

5. FICHIERS DART — SPRINT 1 LIVRÉ

5.1 Fichiers créés et validés

Fichier Dart	Rôle	Correspondance Swift
`models/salaire.dart`	Classe Salaire — tous les champs du calcul (brut, net, cotisations, impôts, crédits)	`Salaire.swift`
`logic/cls_maths.dart`	myRound() arrondi commercial + roundUp() arrondi supérieur	`clsMaths.swift`
`logic/bareme_impot.dart`	Tranches fiscales 2017/2024/2025 pour les 3 classes (if séquentiels)	`BaremeImpot.swift`
`logic/impots.dart`	Calcul impôt brut + contribution de solidarité (7% ou 9%)	`Impots.swift`
`logic/calcul_salaire.dart`	Moteur principal : cotisations, dépendance, CISSM, CI, net final	`CalculSalaire.swift`
`logic/calcul.dart`	Façade : calculBrutNet() et calculNetBrut() itératif	`Calcul.swift`

5.2 Placement dans le projet Flutter

```
ton_projet_flutter/
└── lib/
    ├── models/
    │   └── salaire.dart          ← copier ici
    └── logic/
        ├── cls_maths.dart        ← copier ici
        ├── bareme_impot.dart     ← copier ici
        ├── impots.dart           ← copier ici
        ├── calcul_salaire.dart   ← copier ici
        └── calcul.dart          ← copier ici
```

✓ Aucune dépendance externe requise pour le Sprint 1. Les 6 fichiers fonctionnent avec flutter:sdk seul.

6. PLAN DE DÉVELOPPEMENT — 5 SPRINTS

1 Sprint 1 — Socle — modèles + logique pure

Tout ce qui calcule, sans aucune UI. Peut être testé en ligne de commande.

~2h

- `models/salaire.dart`
- `models/constantes.dart`
- `logic/cls_maths.dart`
- `logic/bareme_impot.dart`
- `logic/impots.dart`
- `logic/calcul_salaire.dart`
- `logic/calcul.dart`

Calcul Brut→Net opérationnel — validé à 2834.68€ ✓

2 Sprint 2 — MVP visible — premier écran + résultats

L'app tourne, on peut saisir un brut et voir un net. Pas beau, mais fonctionnel.

~3h

À FAIRE

- `main.dart`
- `providers/simulation_provider.dart`
- `screens/brut_net_screen.dart`
- `screens/resultat_screen.dart`
- `widgets/result_card.dart`

App installable sur téléphone

3 Sprint 3 — Features cœur — historique + comparateur

Valeur différenciante : sauvegarde locale, slider temps réel, comparateur multi-années.

~4h

À FAIRE

- `services/storage_service.dart`
- `models/simulation.dart`
- `providers/historique_provider.dart`
- `screens/historique_screen.dart`
- `widgets/salary_slider.dart`
- `widgets/comparateur_annees.dart`
- `screens/net_brut_screen.dart`

App démo-ready pour Proximus NXT

4	**Sprint 4 — Export + polish UI** PDF brandé Proximus, écran paramètres, optimisateur avantages en nature.	~3h	**À FAIRE**
	– `services/pdf_service.dart` – `widgets/export_pdf_button.dart` – `screens/parametres_screen.dart`	**App publiable sur App Store / Play Store**	

5	**Sprint 5 — Services — API Apsal + Analytics KPIs** Connecteur API Apsal optionnel + Firebase Analytics anonymisé + dashboard BAs.	~4h	**À FAIRE**
	– `services/api_apsal.dart` – `services/analytics_service.dart` – `providers/params_provider.dart` – `screens/analytics_screen.dart`	**Données d'usage disponibles pour les BAs Proximus**	

7. LES 3 AXES DE FEATURES

iApsal reste une application STANDALONE. L'API Apsal est un connecteur optionnel. Les 3 axes sont ordonnés par priorité business.

AXE 1 — Valeur immédiate

Simulateur de négociation temps réel Slider brut→net mis à jour à chaque mouvement. Aucun appel réseau. Idéal pour entretien ou réunion RH. StatefulWidget + calcul Dart local.	**Standalone**
Comparateur multi-années 2024/2025/2026 Trois colonnes calculées simultanément depuis les barèmes embarqués. Montre l'impact des réformes CCSS sans configuration.	**Standalone**
Synchronisation paramètres légaux Si l'utilisateur connecte son compte Apsal, les taux sont mis à jour automatiquement. Sinon, les valeurs embarquées restent actives. GET /api/v1/parametres/{annee}	**API optionnelle**
Optimisateur avantages en nature Simule l'impact fiscal d'une voiture de fonction, chèques-repas, bonus. Répond à « vaut-il mieux X€ brut ou Y€ d'avantages ? »	**Standalone**

AXE 2 — Rétention utilisateur

Historique local des simulations Chaque calcul sauvegardé automatiquement. Nommage libre, comparaison. Rien ne quitte l'appareil. Package : sqflite ou Hive.	**Standalone**
Export PDF fiche salariale brandée PDF avec logo Proximus NXT partageable. Chaque PDF distribué = visibilité produit. Packages : pdf + printing + share_plus.	**Standalone**
Notifications légales push Proximus envoie une notification quand un taux change (indice, CMP, plafond). Package : firebase_messaging.	**API optionnelle**

Mode multi-profils Plusieurs profils (conjoint, collègue, simulation) sans compte utilisateur. Switching instantané. Hive boxes nommées par profil.	**Standalone**

AXE 3 — KPIs Business Analysts Proximus

iApsal devient un panel comportemental en temps réel sur les attentes salariales luxembourgeoises. Données anonymisées, RGPD compliant. Stack : Firebase Analytics ou PostHog self-hosted.

KPI	Signal marché capté	Priorité
Tranches de simulation	Distribution des bruts simulés par paliers de 500€ → fourchettes les plus recherchées	**Élevée**
Taux usage Net→Brut	% candidats qui négocient à partir d'un net → indicateur tension salariale	**Élevée**
Classes d'impôt actives	Répartition cl.100/200/300 → proxy structure familiale base utilisateurs	Moyenne
Fréquence comparateur années	Corrélé aux changements légaux → impact perçu des réformes CCSS	Moyenne
Sessions avant export PDF	Nombre de simulations avant export → indicateur maturité de décision	Moyenne
Région géographique	LU / FR / DE / BE → répartition frontaliers vs résidents	**Élevée**
Uptake connexion API Apsal	% iApsal → Apsal → taux de conversion produit sans CTA intrusif	**Élevée**

8. PUBSPEC.YAML — DÉPENDANCES COMPLÈTES

```
dependencies:
  flutter:
    sdk: flutter

  # State management
  provider: ^6.1.0

  # Stockage local (historique, profils)
  hive: ^2.2.3
  hive_flutter: ^1.1.0

  # Export PDF + partage
  pdf: ^3.10.8
  printing: ^5.12.0
  share_plus: ^7.2.1

  # Réseau (API Apsal optionnelle)
  http: ^1.2.0

  # Analytics anonymisé (Axe 3 — KPIs BAs)
  firebase_analytics: ^10.8.0
  firebase_core: ^2.27.0

dev_dependencies:
  flutter_test:
    sdk: flutter
  hive_generator: ^2.0.1
  build_runner: ^2.4.8
```

8.1 Ordre d'installation par sprint

Sprint	Package	Usage
1	`Aucun`	Logic pure — flutter:sdk suffisant
2	`provider: ^6.1.0`	State management pour l'UI
3	`hive + hive_flutter`	Stockage local historique/profils
4	`pdf + printing + share_plus`	Export PDF brandé Proximus
5	`http + firebase_analytics + firebase_core`	API Apsal + analytics KPIs

9. ITEMS PENDANTS — À COMPLÉTER

9.1 Logique métier

- Compléter CalculCISSM() — vérifier cas limites pour mntBrutCISSM exactement = 3000 et = 3600
- Compléter barèmes classes 200 et 300 pour toutes les années (bareme2025, bareme2024, bareme2017 — déjà dans le code mais vérifier les tranches frontières)
- Implémenter CalculCIM() — crédit d'impôt monoparental (formule non fournie dans les sources Swift)
- Valider les HMC de la table statique pour 2027 quand les dates seront connues

9.2 Écrans manquants (Sprint 2-3)

- net_brut_screen.dart — copie de brut_net_screen avec dNet comme entrée, dBrut initialisé à dNet×1.3
- apropos_screen.dart — logo Proximus NXT, 3 produits Apsal/Gesall/Facturation, liens vidéos YouTube (videosList.swift)
- resultat_screen.dart — Slider de simulation temps réel (identique à l'iOS)
- Écran paramètres — gestion des profils et du connecteur API Apsal

9.3 Tests unitaires

```
// À créer dans test/calcul_test.dart
void main() {
  test('Brut 3500 → Net 2834.68', () {
    final s = Salaire();
    s.iAnnee = 2026; s.iMois = 1; s.dBrut = 3500;
    s.iclasse = 100; s.iMutualite = 0; s.dHMC = 173;
    s.bCalculCI = true; s.swTxImpots = false;
    Calcul().calculBrutNet(s);
    expect(s.dNet, equals(2834.68));  // exact bit pour bit
  });

  test('Net 4000 → Brut 5668.29', () {
    final s = Salaire();
    s.iAnnee = 2026; s.iMois = 1; s.dNet = 4000;
    s.dBrut = 4000; // point de départ
    s.iclasse = 100; s.iMutualite = 0; s.dHMC = 173;
    s.bCalculCI = true; s.swTxImpots = false;
    Calcul().calculNetBrut(s);
    expect(s.dBrut, equals(5668.29));  // exact bit pour bit — myRound post-
boucle
  });
}
```

10. FICHE TECHNIQUE RÉSUMÉ

RÉCAPITULATIF PROJET iApsal FLUTTER
Application : iApsal — Calculateur de salaires luxembourgeois
Client : Proximus NXT Luxembourg
Migration : Swift iOS → Flutter Dart (cross-platform iOS + Android)
Architecture : 5 couches : models / logic / providers / screens / services
Pattern : Provider (state management) + Hive (storage) + couches séparées
Précision : EXACTE — résultats identiques bit pour bit à l'app iOS originale. Tolérance : zéro.
Bug CISSM corrigé : Int(val)×1000 au lieu de Int(val×1000) — différence critique
Bug NetBrut corrigé : Paliers fins +1, +0.5, +0.25, +0.10, +0.05, +0.01 ajoutés
Sprint 1 statut : 6 fichiers Dart livrés et validés (models + logic)
Prochaine étape : Sprint 2 : main.dart + providers + premier écran UI
Dépendances Sprint 1 : Aucune — flutter:sdk uniquement
Tests cobaye obligatoires : calculBrutNet(3500) → 2834.68 | calculNetBrut(4000) → 5668.29
Langue barèmes : Barèmes 2017, 2024, 2025 (utilisé pour 2026) — classes 100/200/300
HMC Secu : Table statique précalculée 2022–2026 dans calcul_salaire.dart
API Apsal : Connecteur OPTIONNEL — GET /api/v1/parametres/{annee} — jamais bloquant
Analytics : Firebase anonymisé — aucune donnée personnelle — RGPD compliant